Compulsory Assignment 5: Sentiment Analysis

Please fill out the the group name, number, members and optionally the name below.

Group number: 3
Group member 1: Sara Marie Åseng Almdahl
Group member 2: Fredrik Gard Haugen
Group member 3: Kim Andre Nielsen
Group name (optional):

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we reccomend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to analyse the sentiment using a RNN and BERT models.

NOTE: It is recommended to use Google Colab with GPU enabled for CA5 tasks. Keras and TensorFlow (version ≥2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA5

To work on the CA5 assignment in Google Colab, follow these steps:

---

1. Connect Google Colab to Google Drive

Your CA5 files should be stored in Google Drive so Colab can access them.
Run the following code to mount your Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
project_path = "/content/drive/MyDrive/Colab Notebooks/DAT300 gruppe3/CA5" #change this to your folder.
os.chdir(project_path)
print("Current working directory:", os.getcwd())
```

```
Current working directory: /content/drive/.shortcut-targets-by-id/1JhNE24MLV8FyDY-nFxrY-bbE_gBYcuLi/DAT300 gruppe3/CA5
```

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime → Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

Note: I recommend using the regular runtime first, as your GPU usage is limited. Write and test your code in the regular runtime to ensure everything works correctly before switching to the GPU for model training.

Introduction

In this assignment, you will analyze whether a movie review is positive or negative. In Task 1, you will use a Recurrent Neural Network (RNN) to perform the analysis. In Task 2, you will be introduced to the encoder block of the Transformer architecture, commonly known as BERT.

Finally, you will compare the results from both approaches and draw a conclusion.

Note: BERT stands for "Bidirectional Encoder Representations from Transformers"

About the Dataset

The IMDb dataset is a collection of 50,000 reviews from the Internet Movie Database (IMDb). The reviews are labeled as either positive or negative.

The IMDb dataset is a binary sentiment analysis dataset for natural language processing or text analytics.

IMDb is a rich source of film data that includes cast and crew lists, movie release dates, box office information, plot summaries, trailers, actor and director biographies, and other trivia. Information on IMDb comes from a variety of sources, such as filmmakers, film studios, on-screen credits, and other official sources.

Task 0: Load and Inspect the Data

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

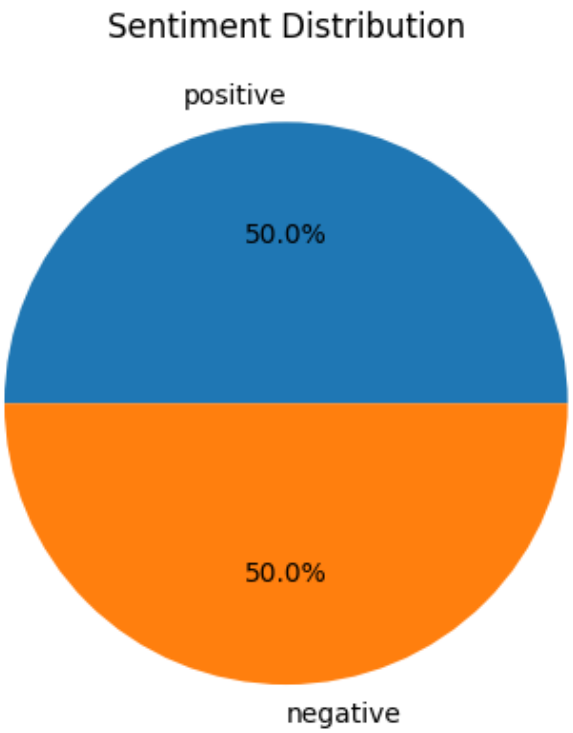
```
data = pd.read_csv("IMDB Dataset.csv")
```

```
data.head()
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
sentiment_counts = data["sentiment"].value_counts()
plt.pie(sentiment_counts, labels=sentiment_counts.index, autopct='%1.1f%%')
plt.title("Sentiment Distribution")
```

```
plt.show()
```



Preprocess

```
from sklearn.preprocessing import LabelEncoder
import re
from sklearn.model_selection import train_test_split
```

```
labels = data["sentiment"]
enc = LabelEncoder()
labels_encoded = enc.fit_transform(labels)
data["labels_encoded"] = labels_encoded
```

```
data.head()
```

	review	sentiment	labels_encoded
0	One of the other reviewers has mentioned that ...	positive	1
1	A wonderful little production. The...	positive	1
2	I thought this was a wonderful way to spend ti...	positive	1
3	Basically there's a family where a little boy ...	negative	0
4	Petter Mattei's "Love in the Time of Money" is...	positive	1

```
def clean_text(text):
    text = re.sub(r"<br\s*/?>", " ", text)  # remove <br> or <br />
    text = re.sub(r" ", " ", text)
    text = re.sub(r"http\S+", " ", text)    # remove URLs
    text = re.sub(r"@\w+", " ", text)       # remove @mentions
    text = re.sub(r"[^A-Za-z0-9\s.,!?']", " ", text)# strip symbols
    text = re.sub(r"\s+", " ", text).strip()
    return text

data["review"] = data["review"].apply(clean_text)
```

```
X_train, X_test, y_train, y_test = train_test_split(data["review"], data["labels_encoded"], test_size=0.25)
```

Task 1: RNN

1.1 Vectorize inputs

```
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization
from tensorflow.keras import layers, Sequential
```

```
max_tokens = 30000      # vocab size cap
max_len    = 128        # sequence length
vectorizer = TextVectorization(
    ################Your Code Here#################
    max_tokens = max_tokens,
```

```
    output_sequence_length = max_len
    ################Your Code Here#################
)


vectorizer.adapt(tf.data.Dataset.from_tensor_slices(X_train).batch(1024))

X_train_vec = vectorizer(X_train)
X_test_vec = vectorizer(X_test)
```

Task 1.1 Create a simple RNN and an LSTM model

```
# RNN
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, LSTM

model_RNN = Sequential()
#model_RNN.add(vectorizer)
model_RNN.add(layers.Embedding(input_dim=max_tokens, output_dim=20, input_length=max_len))
model_RNN.add(layers.SimpleRNN(50, return_sequences=False))
model_RNN.add(layers.Dense(1, activation='sigmoid'))

# Build the model
model_RNN.build(input_shape=(None, max_len))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just r
  warnings.warn(
```

```
# LSTM

model_LSTM = Sequential()
#model_LSTM.add(vectorizer)
model_LSTM.add(Embedding(input_dim=max_tokens, output_dim=20, input_length=max_len))
# Single LSTM layer with 50 units
model_LSTM.add(LSTM(50, return_sequences=False))

# Output layer
model_LSTM.add(Dense(1, activation='sigmoid'))

# Build the model
model_LSTM.build(input_shape=(None, max_len))
```

Compile and print out summary

```
# RNN
model_RNN.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model_RNN.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 128, 20)	600,000
simple_rnn (SimpleRNN)	(None, 50)	3,550
dense (Dense)	(None, 1)	51

Total params: 603,601 (2.30 MB)
Trainable params: 603,601 (2.30 MB)
Non-trainable params: 0 (0.00 B)

```
#LSTM
model_LSTM.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model_LSTM.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 128, 20)	600,000
lstm (LSTM)	(None, 50)	14,200
dense_1 (Dense)	(None, 1)	51

```
Total params: 614,251 (2.34 MB)
Trainable params: 614,251 (2.34 MB)
Non-trainable params: 0 (0.00 B)
```

Task 1.2: Train your model

```
# RNN
model_RNN.fit(X_train_vec, y_train,
              epochs=25,
              batch_size=100)
```

```
Epoch 1/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 14s 16ms/step - accuracy: 0.5127 - loss: 0.6939
Epoch 2/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 18s 24ms/step - accuracy: 0.6476 - loss: 0.6346
Epoch 3/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 11s 30ms/step - accuracy: 0.7524 - loss: 0.5090
Epoch 4/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 9s 24ms/step - accuracy: 0.8707 - loss: 0.2990
Epoch 5/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 7s 19ms/step - accuracy: 0.8887 - loss: 0.2204
Epoch 6/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - accuracy: 0.9203 - loss: 0.1557
Epoch 7/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9413 - loss: 0.1123
Epoch 8/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.9470 - loss: 0.1038
Epoch 9/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9555 - loss: 0.0886
Epoch 10/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9605 - loss: 0.0889
Epoch 11/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9728 - loss: 0.0648
Epoch 12/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9793 - loss: 0.0544
```

```
Epoch 13/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9864 - loss: 0.0374
Epoch 14/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9859 - loss: 0.0383
Epoch 15/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9848 - loss: 0.0382
Epoch 16/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9916 - loss: 0.0229
Epoch 17/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9956 - loss: 0.0143
Epoch 18/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9935 - loss: 0.0206
Epoch 19/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9963 - loss: 0.0120
Epoch 20/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9955 - loss: 0.0135
Epoch 21/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9968 - loss: 0.0105
Epoch 22/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9619 - loss: 0.1123
Epoch 23/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 13s 36ms/step - accuracy: 0.9829 - loss: 0.0467
Epoch 24/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9936 - loss: 0.0201
Epoch 25/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9948 - loss: 0.0161
<keras.src.callbacks.history.History at 0x78b9617175f0>
```

```
# LSTM
model_LSTM.fit(X_train_vec, y_train,
               epochs=25,
               batch_size=100)
```

```
Epoch 1/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.6170 - loss: 0.6139
Epoch 2/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.8875 - loss: 0.3006
Epoch 3/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9146 - loss: 0.2428
Epoch 4/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9313 - loss: 0.1981
Epoch 5/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.8216 - loss: 0.3842
Epoch 6/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.6550 - loss: 0.6227
Epoch 7/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.7036 - loss: 0.6065
```

```
Epoch 8/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.7145 - loss: 0.5603
Epoch 9/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.8453 - loss: 0.3802
Epoch 10/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9039 - loss: 0.2570
Epoch 11/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9304 - loss: 0.2059
Epoch 12/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9475 - loss: 0.1613
Epoch 13/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9621 - loss: 0.1193
Epoch 14/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9697 - loss: 0.1030
Epoch 15/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9755 - loss: 0.0869
Epoch 16/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9784 - loss: 0.0779
Epoch 17/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9819 - loss: 0.0679
Epoch 18/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9860 - loss: 0.0567
Epoch 19/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9863 - loss: 0.0573
Epoch 20/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9886 - loss: 0.0486
Epoch 21/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9869 - loss: 0.0518
Epoch 22/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9880 - loss: 0.0472
Epoch 23/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9885 - loss: 0.0453
Epoch 24/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9903 - loss: 0.0383
Epoch 25/25
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9921 - loss: 0.0343
<keras.src.callbacks.history.History at 0x78b96172acc0>
```

Task 1.3 Evaluate your models

```
from sklearn.metrics import accuracy_score
import numpy as np

y_pred_RNN = model_RNN.predict(X_test_vec, verbose=0)
y_pred_RNN = np.where(y_pred_RNN > 0.5, 1, 0)
```

```
accuracy_RNN = accuracy_score(y_test, y_pred_RNN)

y_pred_LSTM = model_LSTM.predict(X_test_vec, verbose=0)
y_pred_LSTM = np.where(y_pred_LSTM > 0.5, 1, 0)
accuracy_LSTM = accuracy_score(y_test, y_pred_LSTM)

print(f"RNN Accuracy:\t\t{accuracy_RNN:.3g}")
print(f"LSTM tAccuracy:\t\t{accuracy_LSTM:.3g}")
```

```
RNN Accuracy:           0.562
LSTM tAccuracy:         0.838
```

Task 2: BERT

Task 2.0: Install the trasformers package from HuggingFace

Hugging Face is an American company and an open-source community that provides tools, pre-trained machine learning models, and datasets, primarily focused on natural language processing (NLP) and other AI fields like computer vision and audio

Note: You may need to restart the runtime after installing the packages. Then, mount your Google Drive and rerun Task 0.

```
!pip install -U "tensorflow>=2.17,<2.21" "keras>=3.3.0" "tf-keras>=2.17" "transformers>=4.44.0"
```

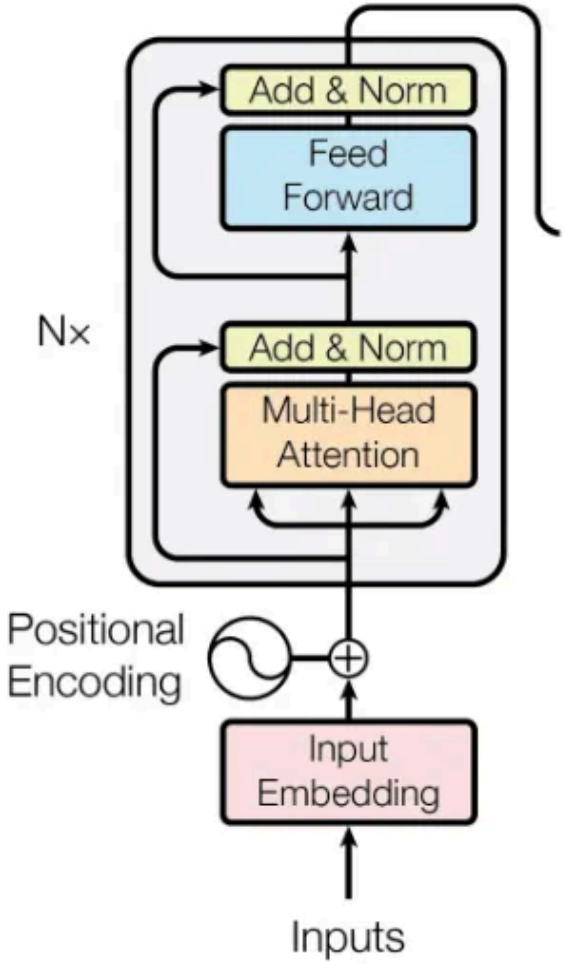
```
Requirement already satisfied: tensorflow<2.21,>=2.17 in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: keras>=3.3.0 in /usr/local/lib/python3.12/dist-packages (3.12.0)
Requirement already satisfied: tf-keras>=2.17 in /usr/local/lib/python3.12/dist-packages (2.20.1)
Requirement already satisfied: transformers>=4.44.0 in /usr/local/lib/python3.12/dist-packages (4.57.3)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.1
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (25.0)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (5.29.5)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (3.2.0)
```

```
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (1.76.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (2.20.0)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (3.15.1)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.21,>=2.17) (0.5.4)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.3.0) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.3.0) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.3.0) (0.18.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (0.36.
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (0.22.1
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.44.0) (4.67.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow<2.21,>=2
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transforme
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transf
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>=2.17)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow<2.21,>
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow<2.21,>=2.
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow<2.21,>=2.17) (11.3
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow<2.21,>=2.
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.3.0) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.3.0) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.3.0) (0.
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->t
```

```
import tf_keras as keras
import tensorflow as tf
from transformers import TFDistilBertForSequenceClassification, TFDistilBertModel, AutoTokenizer
```

Task 2.1: Load the destilbert model

In this part of the assignment, we will use a smaller and faster version of BERT called DistilBERT. It is significantly more lightweight and trains much faster than the original BERT model. Here is the illustration of its architecture:




```
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model =  TFDistilBertForSequenceClassification.from_pretrained(model_name, num_labels=2, from_pt=True)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 665B/s]

config.json: 100%  483/483 [00:00<00:00, 11.0kB/s]

vocab.txt: 100%  232k/232k [00:00<00:00, 1.86MB/s]

tokenizer.json: 100%  466k/466k [00:00<00:00, 3.71MB/s]

pytorch_model.bin: 100%  268M/268M [00:03<00:00, 138MB/s]

TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or pinning yo
Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFDistilBertForSequenceClassification: ['vocab_transfo
- This IS expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model trained on another task or with ano
- This IS NOT expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model that you expect to be exactly i
Some weights or buffers of the TF 2.0 model TFDistilBertForSequenceClassification were not initialized from the PyTorch model and are new
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
def compute_embeddings(encodings):
    input_ids = encodings["input_ids"]
    attention_mask = encodings["attention_mask"]
    outputs = model(input_ids, attention_mask=attention_mask)
    # Take the [CLS] token embedding (first token)
    cls_embeddings = outputs.last_hidden_state[:, 0, :]
    return cls_embeddings
```

Task 2.2: Split the training data to Validation and training

```
X_train_BERT, X_val_BERT, y_train_BERT, y_val_BERT = train_test_split(X_train, y_train, test_size=0.2)
```

Task 2.3: Tokenize

You have to tokenize your inputs again, use the AutoTokenizer that we imported earlier.

```
max_len   = 128
batch_sz  = 64
epochs    = 3

enc_train = tokenizer(
    X_train_BERT.astype(str).tolist(), truncation=True, padding="max_length",
    max_length=max_len, return_tensors="np"
)
enc_val = tokenizer(
    X_val_BERT.astype(str).tolist(), truncation=True, padding="max_length",
    max_length=max_len, return_tensors="np"
)
```

Create datasets

This step wraps your input features (enc_train, e.g., input_ids and attention_mask) and labels (y_train) into a TensorFlow-native streaming dataset.

The TensorFlow Dataset API provides an efficient way to feed data to your model:

- Streams data efficiently from RAM or disk.
- Performs batching, shuffling, and preprocessing in parallel on the CPU.
- Keeps the GPU busy by prefetching the next batch while training.
- Integrates seamlessly with Hugging Face's encoded token dictionaries.

```
ds_train = tf.data.Dataset.from_tensor_slices((dict(enc_train), y_train_BERT))
ds_train = ds_train.map(lambda x, y: (x, tf.cast(tf.squeeze(y), tf.int32))) \
                   .shuffle(10000).batch(batch_sz).prefetch(tf.data.AUTOTUNE)

ds_val = tf.data.Dataset.from_tensor_slices((dict(enc_val), y_val_BERT))
ds_val = ds_val.map(lambda x, y: (x, tf.cast(tf.squeeze(y), tf.int32))) \
               .batch(batch_sz).prefetch(tf.data.AUTOTUNE)
```

Task 2.4: Create and Compile model

```
from tensorflow.keras import layers, models

model.layers[0].trainable = False
model.layers[1].trainable = False

model.compile(
    optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)
model.summary()
```

```
Model: "tf_distil_bert_for_sequence_classification"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 distilbert (TFDistilBertMa  multiple                  66362880
 inLayer)

 pre_classifier (Dense)      multiple                  590592

 classifier (Dense)          multiple                  1538

 dropout_19 (Dropout)        multiple                  0

=================================================================
Total params: 66955010 (255.41 MB)
Trainable params: 1538 (6.01 KB)
Non-trainable params: 66953472 (255.41 MB)
_________________________________________________________________
```

Task 2.5: Train your model

```
model.fit(ds_train,
          epochs=epochs,
          batch_size=batch_sz
          )
```

```
Epoch 1/3
469/469 [==============================] - 160s 284ms/step - loss: 0.6228 - accuracy: 0.6871
Epoch 2/3
469/469 [==============================] - 130s 277ms/step - loss: 0.5548 - accuracy: 0.7366
Epoch 3/3
469/469 [==============================] - 131s 278ms/step - loss: 0.5315 - accuracy: 0.7449
```

```
<tf_keras.src.callbacks.History at 0x781ee19861b0>
```

Task 2.6: Evaluate

```
eval = model.evaluate(ds_val)
```

```
118/118 [==============================] - 33s 269ms/step - loss: 0.4962 - accuracy: 0.7819
```

```
print(f"DistilBERT accuracy:\t\t{eval[1]:.3g}")
```

```
DistilBERT accuracy:            0.782
```

Task 3: Compare

3.1: Compare the results of the two models. What could explain the differences in their performance?

Answer: A LSTM model can maximally "remember" ~(200-300) states, which is more than a simple RNN. This explains why the RNN has an test accuracy of 0.562, while the LSTM had a test accuracy of 0.838. A transformer has the potential to remember far longer sequences than an LSTM. We assume that the lower transformer accuracy of 0.782 is due to the small amount of training done.

3.2: Why did we need to tokenize the inputs again using a different tokenizer in Task 2.3?

Answer: DistilBERT's tokenizer does not use words but "subword"s as tokens (It splits up words)

3.3: Briefly explain the main difference between RNNs and Transformer architectures.

Answer: RNN reads in information one token at a time, and all context is sequezed through the same hidden space. Transformers uses self attention to find the a tokens relation to all other tokens. Processing them in parallell.